

Advanced Natural Language Processing

Syntax and Semantics

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes — Course year 2026-2027



Maastricht University

Traditional Linguistic Pipeline in more Detail

- 
1. Language recognition
 2. Tokenization
 3. Word detection
 4. Sentence detection
 5. Syntax
 - a) Part of Speech (POS)
 - b) Phrases
 - c) Deeper Syntactic Analysis
 6. Semantics
 - a) Meaning of items
 7. Deal with more Complex Linguistic Relations (later) such as pronouns, co-references, negations, etc.
 8. Discourse, intent, action, ...

Teaching note: This is the roadmap for the whole course: tokenization/lexicons (last lecture) → syntax (this lecture) → semantics → discourse (later).

We Discussed in previous lecture: Tokenization & Lexicons

Last course we discussed:

- Tokenization
- Lexicons
- Vocabulary
- Thesauri
- Stemming
- Lemmatization
- Case folding
- Normalization
- Language issues

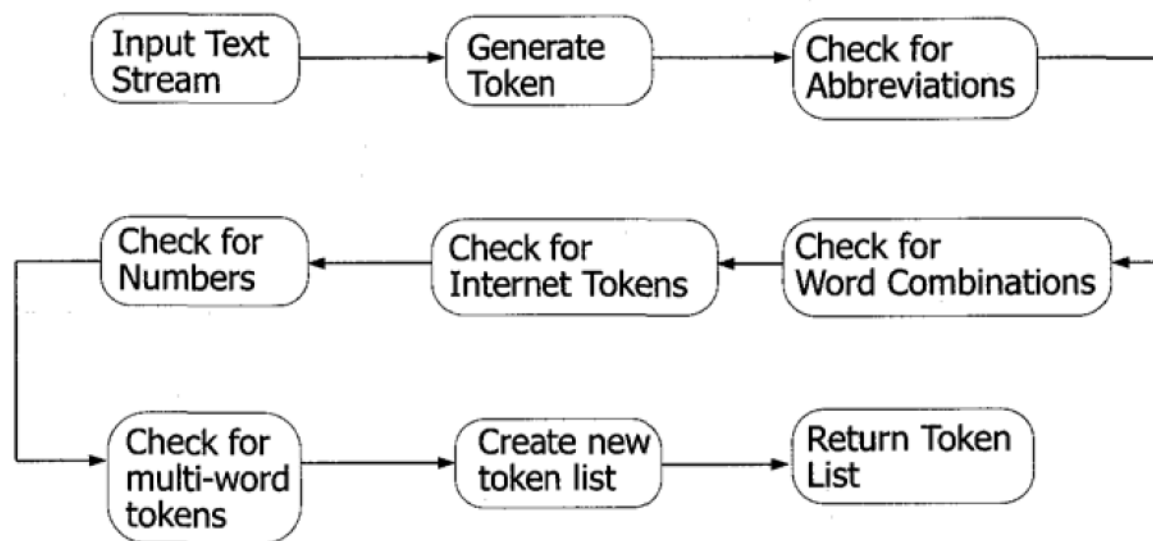


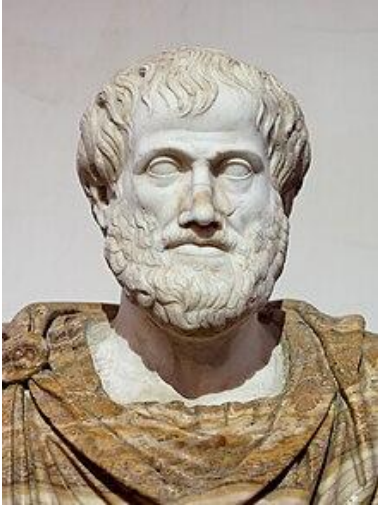
FIGURE 3.1 Steps in the assembly of composite tokens.

https://lucene.apache.org/core/4_0_0/core/org/apache/lucene/analysis/Tokenizer.html

Sentences

- Finding sentences is important.
- One can pretty much rely on punctuation and whitespace alone after proper tokenization (make to not to split on “U.S.” in a sentence.
- But “;” and “:” can cause complications.
- Be aware: not all documents are well-written.
- Not all languages have punctuations.
- Sentences have very clear statistics as well.

Linguistics and syntactic structures



Aristotle (384-322 BC)
Rhetoric, nominal and verbal words in Greek language

Dionysius Thrax (170-90 BC):
Techne Grammatica, “parts of speech”, focus on rhetoric, “good language”

Also early linguistics in India and China



Ferdinand de Saussure (1857 -1913)
Course in General Linguistics

Start of the science of linguistics
Phonology, phonetics, systems of *signs*

Descriptive linguistics

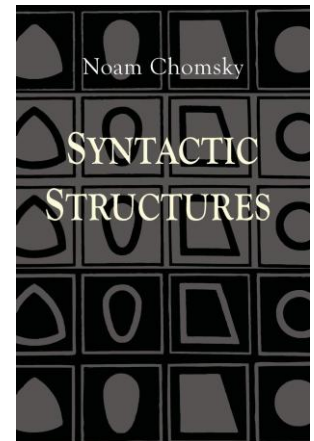


Noam Chomsky (1928-)
Syntactic Structures

Start of the modern linguistics, generative grammars, synthetic languages

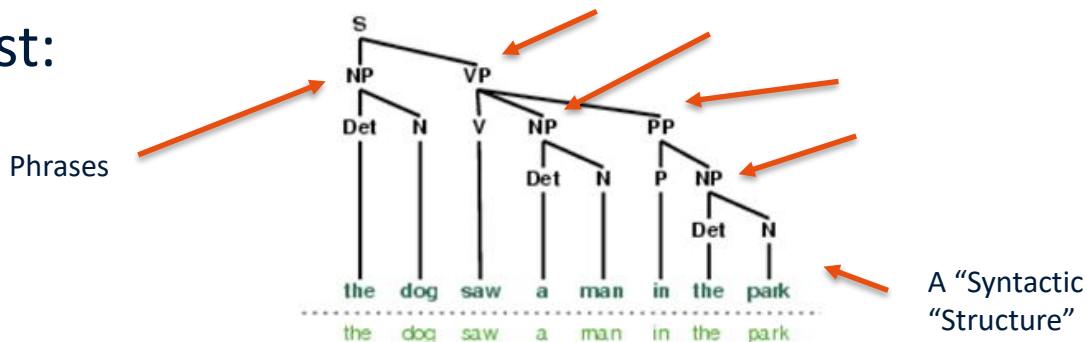
Universal Grammar; the idea of human innate grammatical knowledge

Formal language theory, theoretical computer science (compilers), artificial intelligence



Sentences and Phrases

- A sentence is made up from a string of words and punctuations.
- In a sentence, sub-sentences can be identified: aka phrases.
- Five types of **phrases** exist:
 - Noun
 - Verb
 - Adjective
 - Adverb
 - Prepositional



*She sings **very loudly**.*

- Head = **loudly** (an adverb)
- Whole phrase = **very loudly** → this is an **adverb phrase** (because it's headed by an adverb).

Adverbials

An **Adverbial** is a word (an adverb) or a group of words (an adverbial phrase or an adverbial clause) that modifies or tells us something about the sentence or the verb.

Phrase	Type	Why it's adverbial
very loudly	<i>adverb phrase → adverbial phrase</i>	Tells how she sings
with great enthusiasm	<i>prepositional phrase → adverbial phrase</i>	Also tells how she sings
last night	<i>noun phrase → adverbial phrase</i>	Tells when it happened

Term	Headed by...	Defined by...
Adverb phrase	An adverb	Syntax (headedness)
Adverbial phrase	Any category (adv / NP / PP / clause)	Function (adverbial role)

Object, Predicate (verb), Subject

<i>Sentence</i>	<i>Subject</i>	<i>Verb/Predicate</i>	<i>Object(s)</i>
The dog chased the ball .	The dog	chased	the ball (direct object)
She gave me a gift.	She	gave	me (indirect) + a gift (direct)
He is tired.	He	is tired (predicate; no object)	—

Subject = Noun that the sentence is about. The **Verb** expresses the action or state of being. The **Object** is what receives the action of the verb.

Sentences are made up of adverbials, subject, verb and objects

Example: Yesterday, the students wrote their essays in the library.

<i>Element</i>	<i>Part of the sentence</i>	<i>What it does</i>
Adverbial	Yesterday	Tells <i>when</i> the action happened
Subject	the students	Who performs the action
Verb (predicate)	wrote	The action
Object	their essays	What they wrote (receives the action)
Adverbial	in the library	Tells <i>where</i> the action happened

Objects: direct and in-direct

<i>Object type</i>	<i>What it represents</i>
Direct object	Receives the action (what/whom the verb affects)
Indirect object	Receives the direct object or benefits from the action (to/for whom)

Example:

*She gave **him** a laptop.*

gave **a laptop** → direct object

gave a laptop **to whom?** → **him** → indirect object

POS: Part of Speech

Noun Pronoun Verb Adjective Adverb Conjunction Preposition Article Interjection

Andrew and Maria thought their jobs were secure after the rancorous argument with the customer , but alas ! Bad news is fast approaching them , especially after they viciously insulted the customer on social media .

Part-of-Speech (POS) Tagging: In linguistics, part-of-speech tagging (POS tagging or POST), also called **grammatical tagging** or **word-category disambiguation**, is the process of marking up the words in a text (corpus) as corresponding to a particular part of speech, based on both its definition, as well as its context —ie. relationship with adjacent and related words in a phrase, sentence, or paragraph.

Schools commonly teach that there are 9 parts of speech in English: **noun, verb, article, adjective, preposition, pronoun, adverb, conjunction, and interjection**. However, there are clearly many more categories and sub-categories.

Part of Speech (POS)

For nouns, plural, possessive, and singular forms can be distinguished. In many languages words are also marked for their "case" (role as subject or object), grammatical gender, and so on; while verbs are marked for tense, aspect, and other things.

- In POS, Words can be classified into classes of parts of speech (POS). Eight standard categories exist:
 - Adjectives ("brown", "nice"): modify a noun
 - Adverbs ("good"): modify a verb
 - Conjunctions ("and", "or")
 - Determiners ("the", "a")
 - Nouns ("school")
 - Prepositions ("of", "to", "in", "for")
 - Pronouns ("you", "he", "she"): replace Determine + Noun
 - Verbs ("walk", "learn")
- Sometimes interjections ("*hi*", "*Bye*" and "*Goodbye*") and punctuations are also seen as POS categories.
- There are many subclasses for POS-tags. The Brown Corpus has 87 different POS-tags! The Treebank has 45 different POS-tags.



E.g., a pronoun can be subcategorized into: nominative, accusative, possessive, 2nd possessive, and reflective

POS-2

- **Content words** are: nouns, verbs, adjectives and adverbs. are descriptive in nature and are used to represent people, places, animals, things, and ideas.
- **Functional words** are: conjunctions, determiners, pronouns and prepositions. are grammatical and vocabulary words.

Most words in the lexicon (aka dictionary) are *content* words.

Different Levels of Understanding in NLP: Deel & Shallow

Shallow

- **Tokenization:** break continuous character stream into meaningful constituents. Recognize sentences and words. Extract token features (such as capitalization, inclusion of digits, punctuation, special characters, etc.).
- **Part-of-Speech (POS) Tagging:** annotate the words with appropriate POS tags. At some stage, POS taggers also perform morphological analyses.

Deep

- **Syntactic Parsing:** full syntactical analysis based on a certain grammar theory.
 - **Constituency grammars:** describe syntactical structure with recursive built phrases of sequentially grouped elements. Additionally, syntactic structures have a role: subject, object, etc.
 - **Dependency grammars:** do not recognize constituents as separate linguistic objects but focus on direct relationships between words. These grammars are limited and have problems with conjunctions. However, they are useful dealing with co-references, pronouns and negations.

How can we Describe Syntactic Structures?



Maastricht University

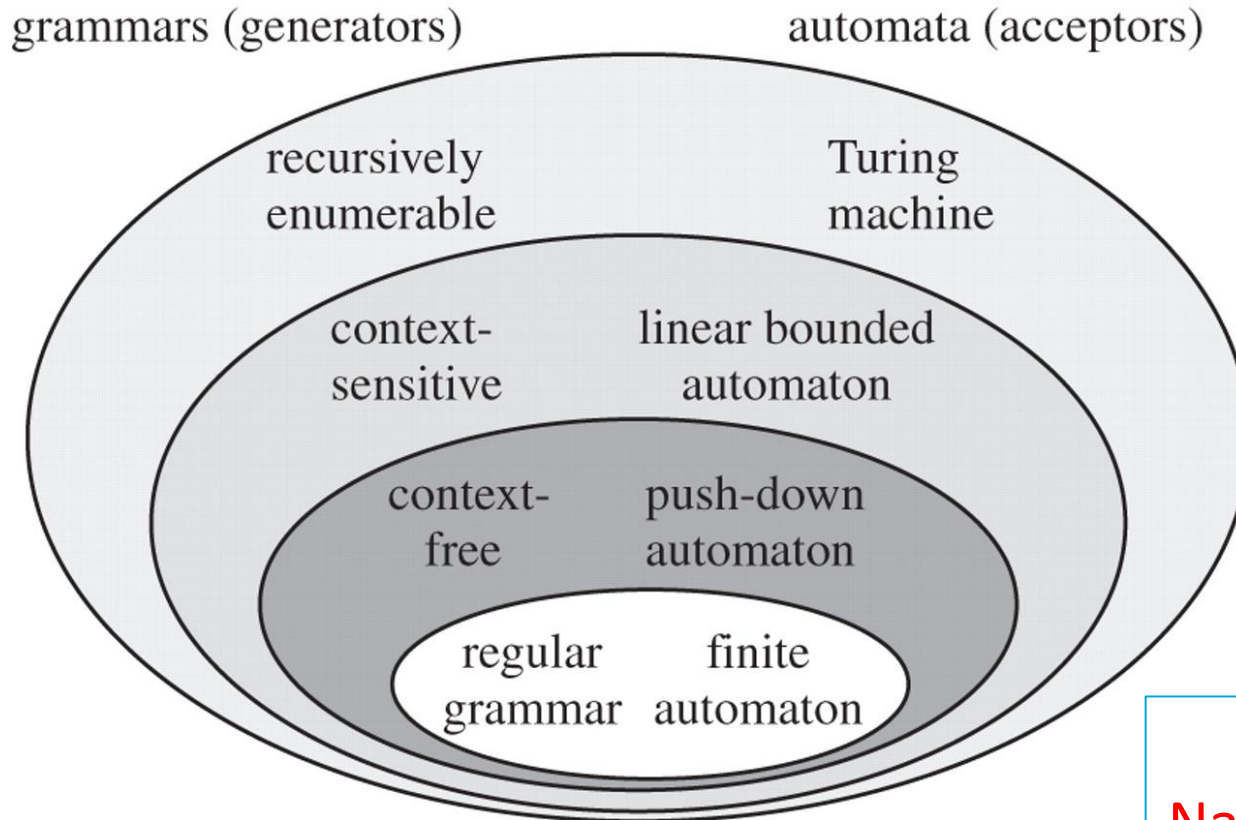
Teaching note: Three equivalent ways to describe a formal language follow: regular expressions, automata, and generative grammars — same power, different notation.

Chomsky Hierarchy: a Theoretical Background

Grammar	Languages	Automaton	Production rules (constraints)*	Examples ^[3]
Type-0	Recursively enumerable	Turing machine	$\gamma \rightarrow \alpha$ (no constraints)	$L = \{w w \text{ describes a terminating Turing machine}\}$
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n n > 0\}$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \alpha$	$L = \{a^n b^n n > 0\}$
Type-3	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$	$L = \{a^n n \geq 0\}$
* Meaning of symbols: • a = terminal • A, B = non-terminal • α, β, γ = string of terminals and/or non-terminals • α, β = maybe empty • γ = never empty				

Source: https://en.wikipedia.org/wiki/Chomsky_hierarchy

Chomsky's hierarchy



Where would
Natural language fit
in this hierarchy?

Describing a (formal) Language

Describing a (formal or computer) language can be done by:

- Regular Expressions, or
- Acceptance Automata, or
- Generative Grammars

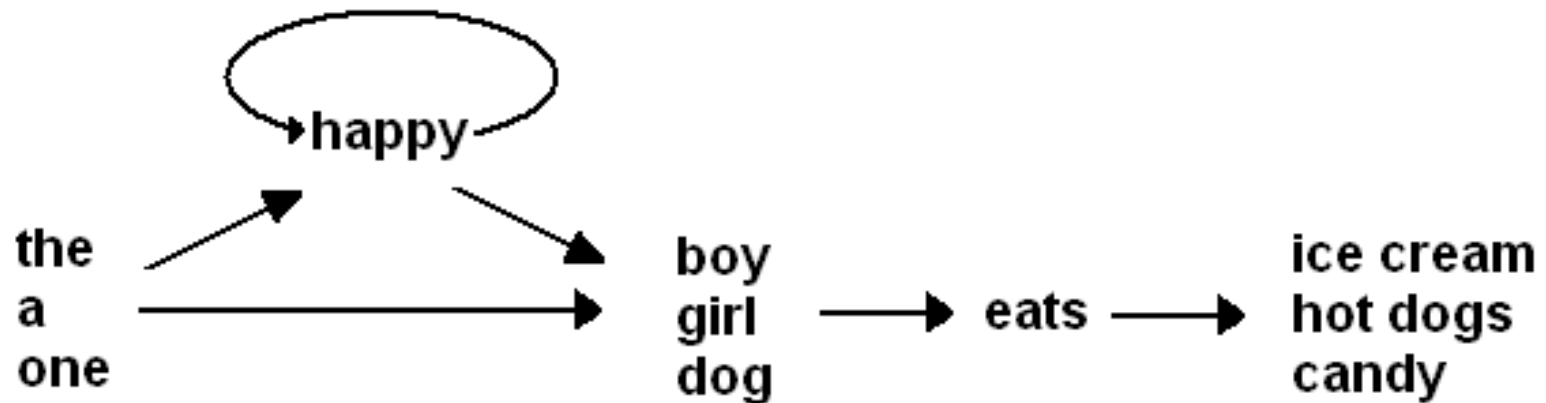
Regular Expressions

- `/a/` = exactly on character “a”
- `/[Ww]/` = one of: either “W” or “w”
- `/[A-Z]/` = one of A,B, ..., Z
- `/[^a]/` = any character, but not an “a”
- `/[^\.]/` = not a dot (escaping)
- `Colou?r` = color or colour
- `/[ab]*/` = zero or more *a*’s and *b*’s
- `/[0-9]+/` = at least one number. String of numbers.
- `/cat|dog/` = disjunction: cat or dog
- `/gupp (y|ies)/` = precedence with ()

Accepting Automata

- A **regular expression** is one way of describing a **finite state automata** (FSA).
- Any regular expression can be implemented as a FSA, as long as the regular expression does not use any memory states.
- Both regular expressions and FSA's can be used to describe a particular kind of formal language: a **regular language**.

Accepting Finite State Machines for a Sentence



Generative Grammar

G_1

$S \rightarrow abS$

$S \rightarrow a$

G_2

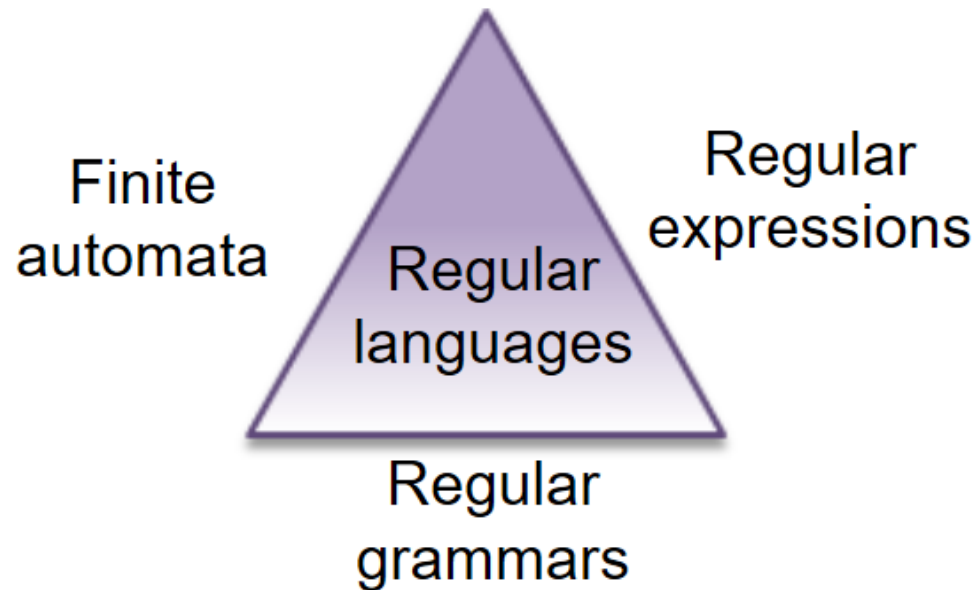
$S \rightarrow Aab$

$A \rightarrow Aab \mid B$

$B \rightarrow a$

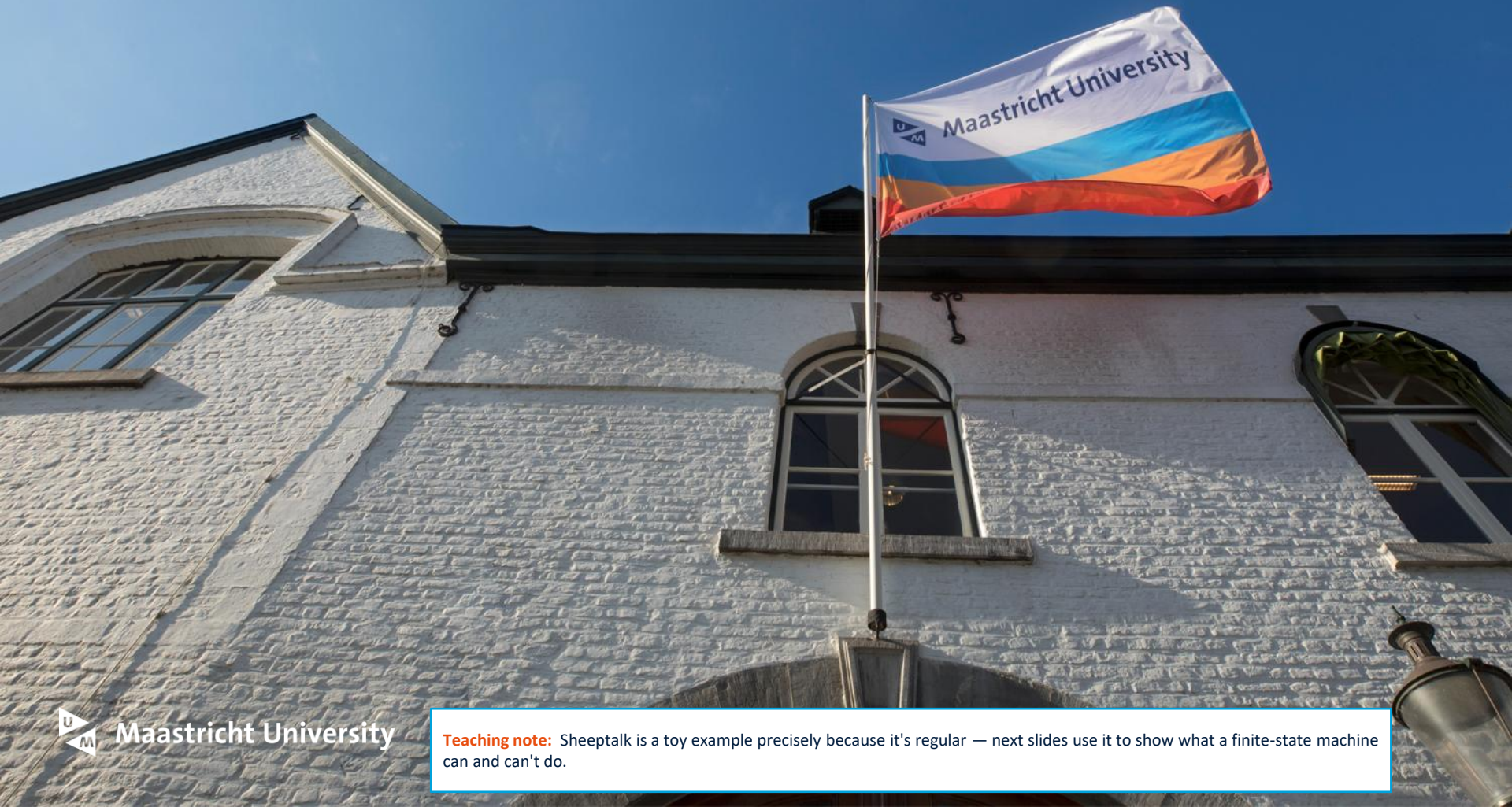
lower-case are end symbols, UPPER-CASE are
rewrite symbols

So, there are 3 ways to describe regular languages



Three equivalent ways of describing regular languages.

A Regular Language: “Sheeptalk”

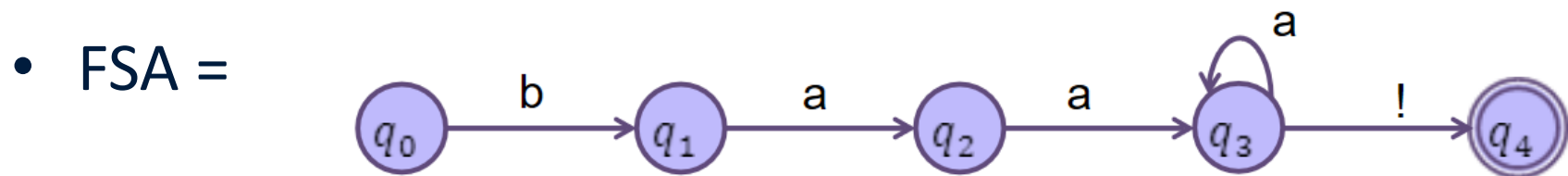


Maastricht University

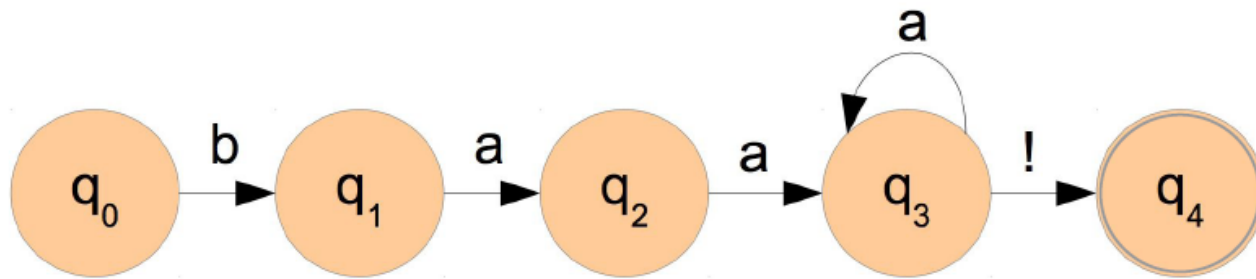
Teaching note: Sheeptalk is a toy example precisely because it's regular — next slides use it to show what a finite-state machine can and can't do.

Language of “sheeptalk”

- Contains of: baa!, baaa!, baaaa!,
- Regular expression = /baa+!/

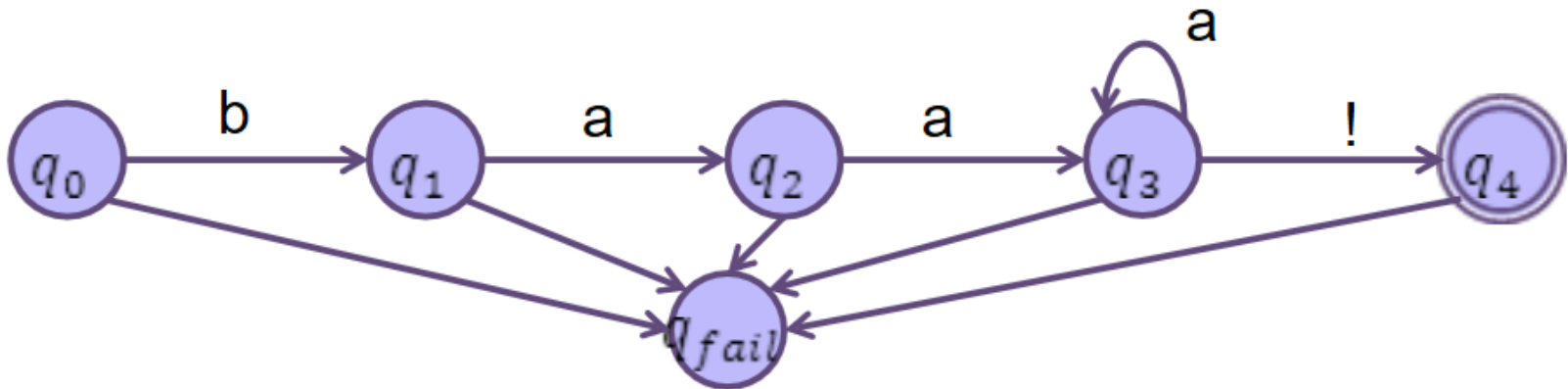


- We have: start state, final or accepting state, and 5 transitions.
- FSA can recognize (accept) or reject strings from the sheeptalk language.



- **Strings accepted:**
 - baa!
 - baaaaa!
- **Strings not accepted:**
 - abc
 - babb
 - !bcd

Adding a fail state



$L(m) = \{ \text{baa!}, \text{baaa!}, \text{baaaa!}, \text{baaaaa!}, \text{baaaaaaa!}, \dots \}$

We can use this to parse a sentence

Formal Languages to Describe or Generate sentence

- A model that can both generate and recognize all and only the strings of a formal language acts as a *definition* of that formal language.
- Model m , then $L(m)$ is the formal language characterized by m .
- $L(m) = \{\text{baa!}, \text{baaa!}, \text{baaaa!}, \dots\}$ for the sheeptalk language.
- A formal language almost always bears no resemblance at all to a natural language.
- But, we use formal languages to model parts of natural language such as phonology, morphology or syntax.
- Generative grammar = a grammar of a formal language.



“Sheeptalk” with a generative grammar

- regular sheeptalk

- $s \rightarrow [b], x.$
- $x \rightarrow [a], y.$
- $y \rightarrow ['!'].$
- $y \rightarrow [a], y.$

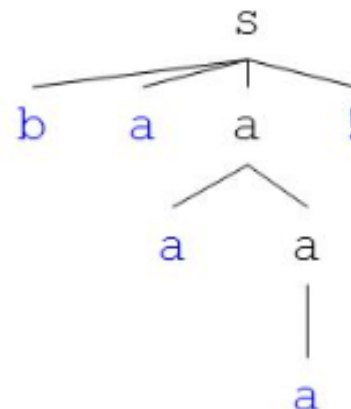
- query

?- $s(X, [])$.

$X = [b, a, !]$? ;

$X = [b, a, a, !]$? ;

$X = [b, a, a, a, !]$? ;



Is Natural Language a
Regular Language?

Finite State and Natural Language

- Why can't we use Finite-State methods to capture all the syntactic facts of a natural language.
- Answers:
 - Certain syntactic structures as they exist in natural language make them not regular languages.
 - Even when FSA are capable of capturing the syntactic structures, the models become so complex and hard to understand. *Recursion then needs to be described as iteration.*

FSA, Regular Grammars & Regular Languages ...

- FSA are equivalent to regular grammars.
- Regular grammars can describe regular languages.
- All rules in a regular grammar are of one of the forms:
 - (i) $A \rightarrow w^*$: non-terminals expand to terminals, or
 - (ii) $A \rightarrow w^*B$: non-terminals expand to non-terminals and terminals.

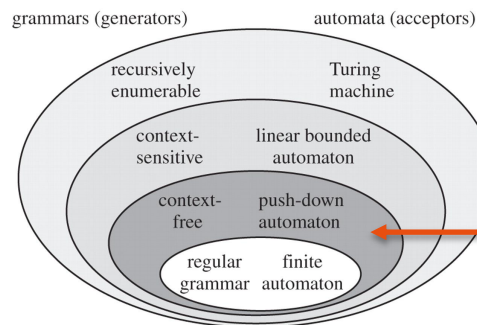
Center-Embedding in Natural Language

“The luggage arrived”

“The Luggage that the passengers checked arrived”

“The luggage that the passengers that the storm delayed checked arrived”

...etc...



This is where we need a more powerful mechanism: Context Free Grammars (CFG)

Natural Language is not a regular language

- Regular grammars cannot express languages that express **recursive center embedding**: a non-terminal is rewritten as itself surrounded by (non-empty) strings: $A \Rightarrow \alpha A \beta$.
- As a result, a language can be generated by a FSA as long as it does not have **center-embedded** recursions.
- Intuitively this is because grammar rules in which the non-terminal symbols are always on the right or on the left edge of a rule can be processed iteratively and not recursively.
- If one models center-embedded recursion with a FSA then it needs to be modeled as an iterative process. This will always result in either a very complex deep process or into limited recursion.

Context Free Grammars for NLP



Maastricht University

Teaching note: CFGs add a stack (memory) that finite-state machines lack — that's exactly what's needed for the center-embedding examples just shown.

Context Free Grammars (CFG)

- CFG are one of the most common use mathematical system for modeling constituent structures English and in other languages.
- **CFG allow recursion.**
- CFG are also known as Phrase-Structure Grammars. Formalist is Backus-Naur form (BNF).
- Ideas of CFG goes back to Wilhelm Wundt (1900), but was not formalized until the work done independently by Chomsky (1959) and Backus (1959).
- CFG = rules (or productions) that express how symbols of a language can be grouped or ordered PLUS a lexicon of words and symbols.

Some example CFG for a Noun Phrase

Nouns can stack (e.g.,
cargo flight schedule
change)

Definitions:

NP (Noun Phrase): A phrase that behaves like a noun.

Det (Determiner): Words like a, the, some, this.

Noun: A lexical noun (e.g., flight, doctor, book).

ProperNoun: A proper name (e.g., Smith, Paris).

Nominal: A noun plus optional additional nouns.

NP $\rightarrow$ Det Nominal

NP $\rightarrow$ ProperNoun

Nominal $\rightarrow$ Noun

Nominal $\rightarrow$ Nominal Noun

Det $\rightarrow$ a

Det $\rightarrow$ the

Noun $\rightarrow$ flight

Noun $\rightarrow$ doctor

ProperNoun $\rightarrow$ Smith

Where is the recursion?

CFG Symbols

- Terminals: symbols that correspond to the words in a language.
- Non-terminals: symbols that express clusters of generalizations.
- In $\text{SYMBOL} \rightarrow \text{SYMBOL}$, left handed is always a single non-terminal. Right handed is an ordered list of one or more terminals and non-terminals.
- CFG's can be used to (i) generate sentences and (ii) to accept, parse and assign a structure to sentences.
- CFG allow for recursion: $A \rightarrow aAb$ or $A \rightarrow aA$, ... right side can be anything (also empty)
- Remember FSG:
 - $A \rightarrow a$
 - $A \rightarrow aB$
 - **NOT ALLOWED:** $A \rightarrow aA$

NOTE: See also last slides of this course on Chomsky's Hierarchy.



An Example of a Lexicon for L_0

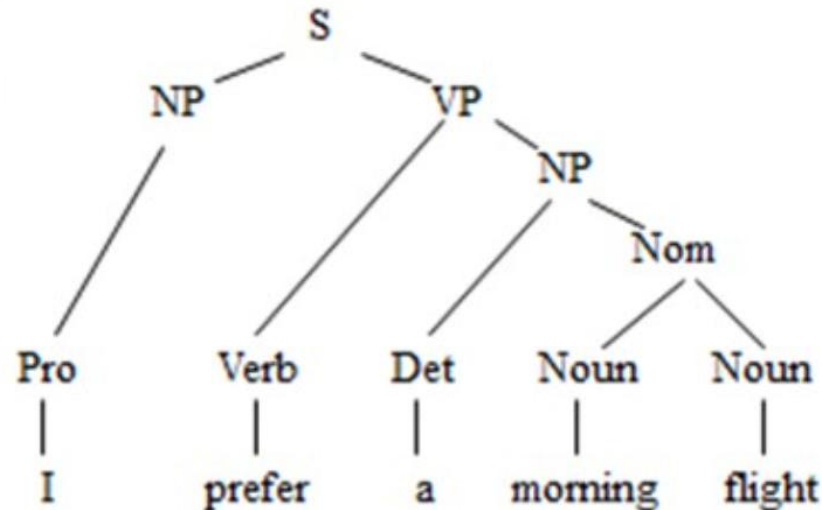
Noun → *flights* | *breeze* | *trip* | *morning*
Verb → *is* | *prefer* | *like* | *need* | *want* | *fly*
Adjective → *cheapest* | *non-stop* | *first* | *latest*
 | *other* | *direct*
Pronoun → *me* | *I* | *you* | *it*
Proper-Noun → *Alaska* | *Baltimore* | *Los Angeles*
 | *Chicago* | *United* | *American*
Determiner → *the* | *a* | *an* | *this* | *these* | *that*
Preposition → *from* | *to* | *on* | *near*
Conjunction → *and* | *or* | *but*

Grammar Rules of L_0

Grammar Rules	Examples
$S \rightarrow NP VP$	I + want a morning flight
$NP \rightarrow$ $Pronoun$ $Proper-Noun$ $Det Nominal$	- I - Los Angeles - a + flight
$Nominal \rightarrow$ $Nominal Noun$ $Noun$	- morning + flight - flights
$VP \rightarrow$ $Verb$ $Verb NP$ $Verb NP PP$ $Verb PP$	- do - want + a flight - leave + Boston + in the morning - leaving + on Thursday
$PP \rightarrow Preposition NP$	from + Los Angeles

A parse Tree for “I prefer a morning flight”. According to grammar L_0

Driven by the grammar, working down



$S \rightarrow NP VP$, $NP \rightarrow Pro$, $Pro \rightarrow I$, $VP \rightarrow V NP$, $V \rightarrow prefer$, $NP \rightarrow Det$
 Nom , $Det \rightarrow a$, $Nom \rightarrow Noun Nom$, $Noun \rightarrow morning$, $Noun \rightarrow flight$

$[_S [_{NP} [_{Pro} I]] [_{VP} [_{V} prefer] [_{NP} [_{Det} a] [_{Nom} [_{N} morning] [_{N} flight]]]]]$

Is Natural Language Context-Free?

- Natural language can be very complex. Do CFG have enough power to capture the entire linguistic complexity? **No**.
- For instance: **cross-serial dependencies** cannot be covered by a CFG. In Dutch, verbs selecting an infinitival complement (e.g. modals and perception verbs) give rise to so called cross-serial dependencies. The arguments of the verbs involved appear in the same order as the verbs in the 'verb cluster':

- (1) dat ik₁ haar₂ de nijlpaarden₂ zag₁ voeren₂
that I her the hippos saw feed
“that I saw her feed the hippos”
- (2) dat ik₁ Henk₂ haar₃ de nijlpaarden₃ zag₁
that I Henk her the hippos saw
helpen₂ voeren₃
help feed
“that I saw Henk help her feed the hippos”



Dependency Grammars

Dependency grammars help us understand how words connect within sentences, much like assembling puzzle pieces. It focuses on word-to-word relationships, using arrows or lines to represent these connections.



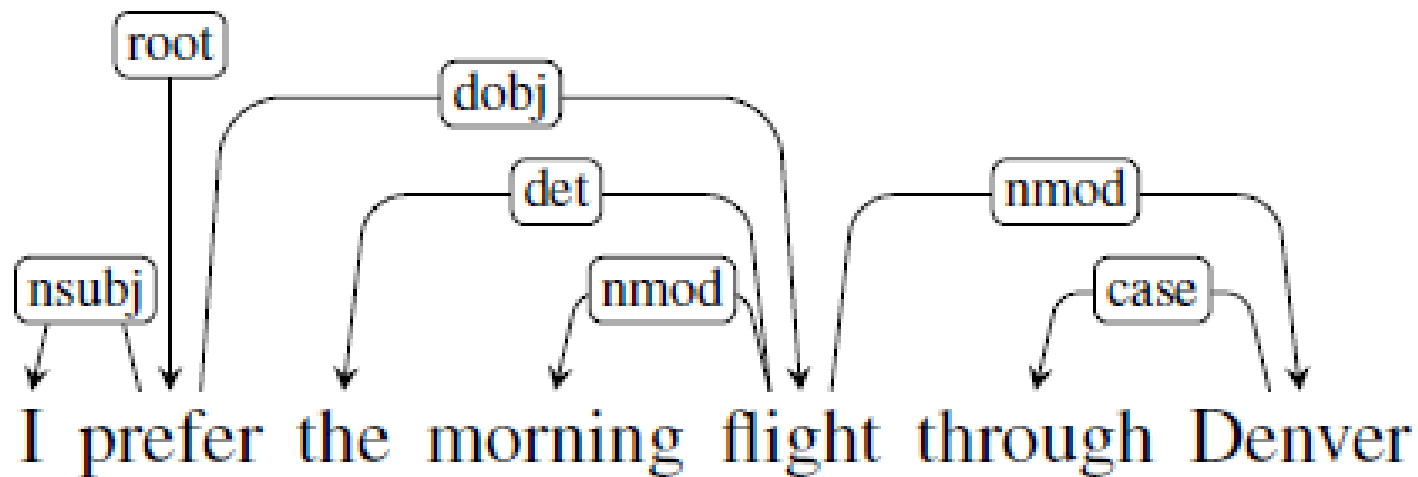
Dependency Grammars

- Dependency grammars are based on work of Tesnière (1959).
- Has much older roots than the CFG theories. Goes even back to ancient Greek and Indian.
- “Parsing a sentence into subject and predicate”.
- Is more based on lexical relations than on constituent relations.
- There are no non-terminal or phrasal nodes in such a parse tree: all relations are between two lexical nodes.

Speech and Language Processing. Daniel Jurafsky & James H. Martin.



Dependency Parse Tree



Relations among the words are illustrated above the sentence with directed, labeled arcs from heads to dependents. We call this a **typed dependency structure** because the labels are drawn from a fixed inventory of grammatical relations. It also includes a *root* node that explicitly marks the root of the tree, the head of the entire structure.

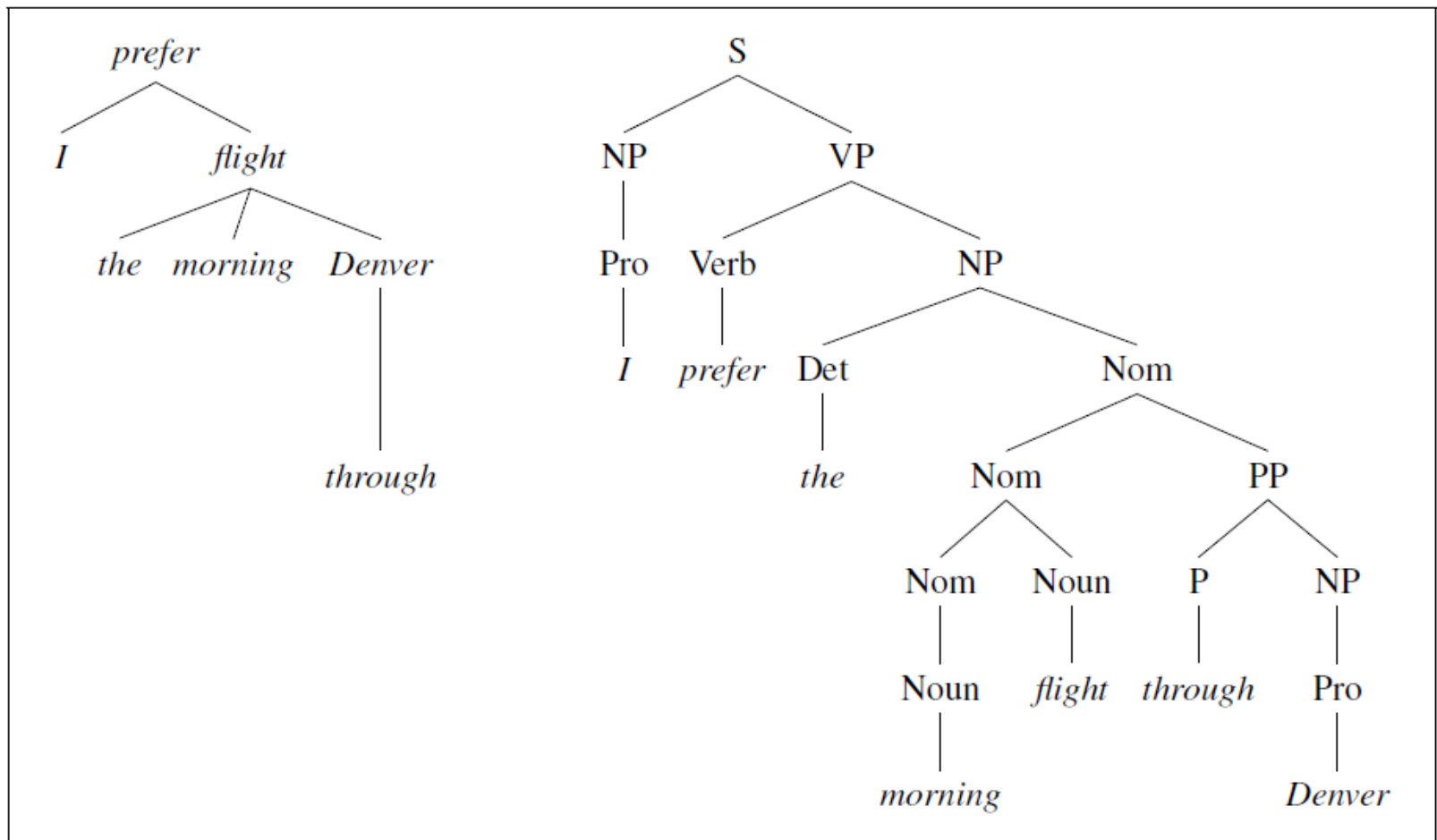


Figure 13.1 A dependency-style parse alongside the corresponding constituent-based analysis for *I prefer the morning flight through Denver*.

DP versus C-PS Parse

Dependency parse is like a family tree of word relationships. (“Who depends on whom?”)

Constituent parse is like a set of nested folders. (“Which words belong together as a phrase?”)

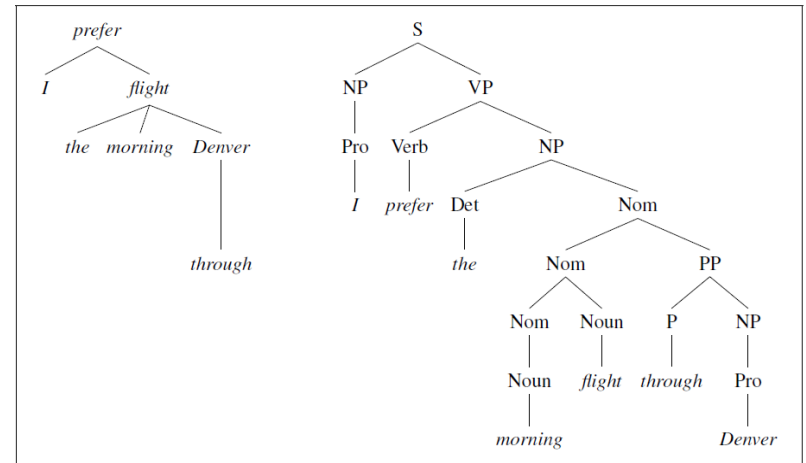


Figure 13.1 A dependency-style parse alongside the corresponding constituent-based analysis for *I prefer the morning flight through Denver*.

Feature

Dependency Parse

Constituent (Phrase-Structure) Parse

Focus

Word-to-word relationships

How words group into phrases

Units

Words only

Words + phrases (NP, VP, PP, etc.)

Representation

Flat, head-based

Hierarchical, phrase-based

Root of sentence

The main verb (e.g., *prefer*)

The sentence node *S*

Used in

Modern NLP parsers (e.g., spaCy, UD)

Formal syntax, grammars, CFGs

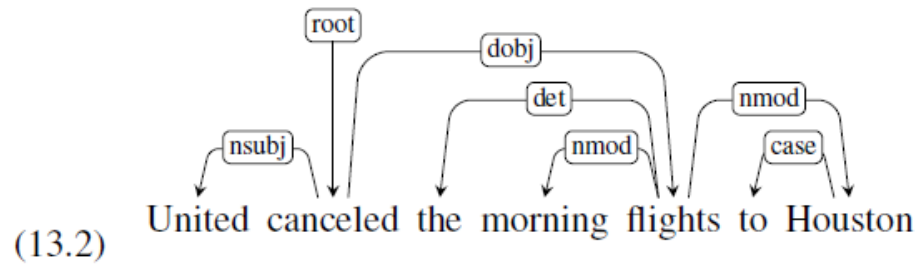
Grammatical Relations in a Dependency Grammar

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Figure 13.2 Selected dependency relations from the Universal Dependency set. (de Marneffe et al., 2014)

Example

Consider the following example sentence:



The clausal relations NSUBJ and DOBJ identify the subject and direct object of the predicate *cancel*, while the NMOD, DET, and CASE relations denote modifiers of the nouns *flights* and *Houston*.

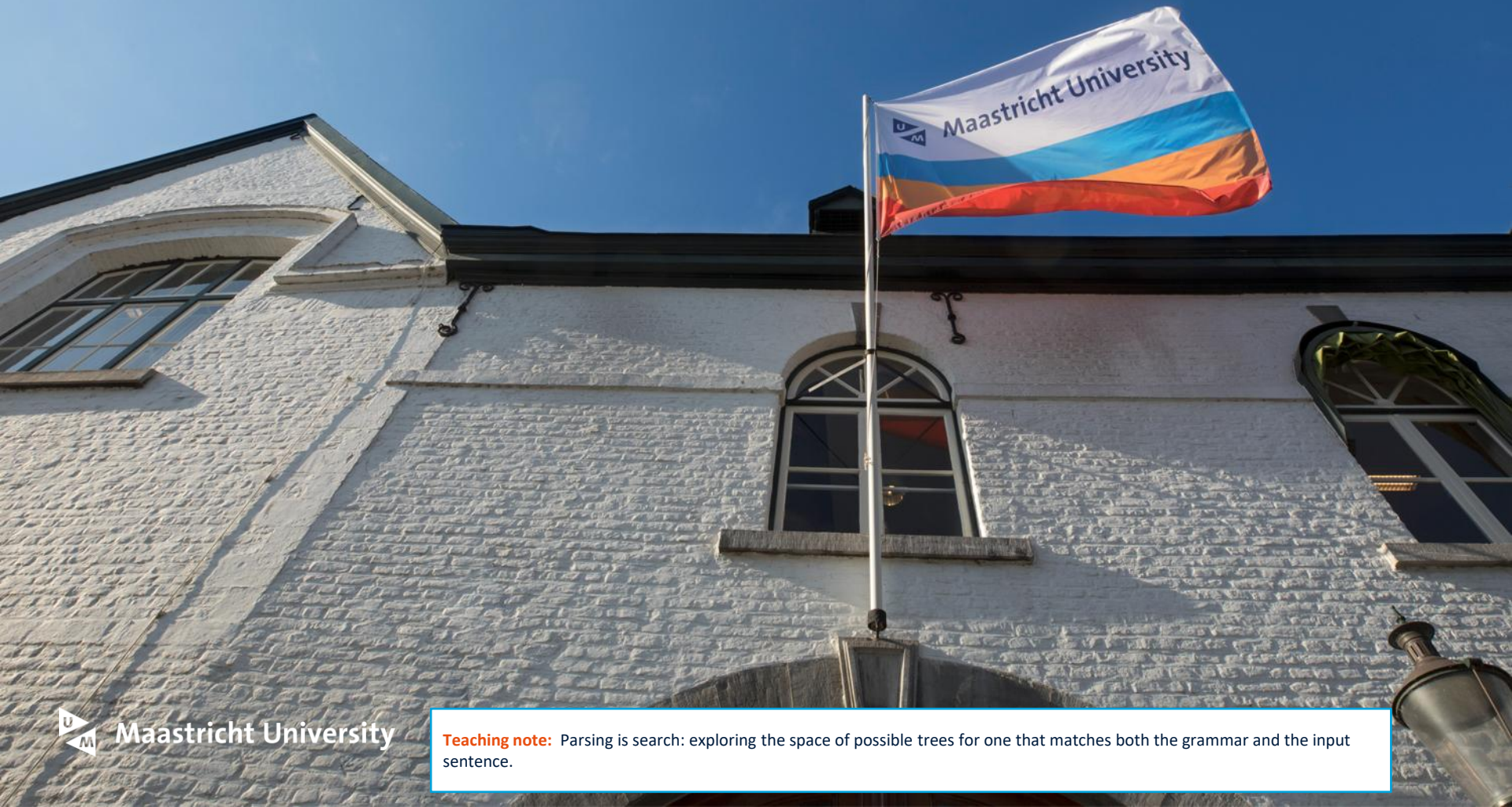
<i>Dependency label</i>	<i>Meaning</i>	<i>Example here</i>
root	Main verb of the sentence	canceled
nsubj	Subject performing the action	United
dobj	Direct object receiving the action	flights
det	Determiner modifying a noun	the
nmod	Nominal modifier (often part of a PP)	morning / Houston
case	The preposition linking a PP to a noun	to

More on Dependency Grammars

- Dependency grammars have a strong predictive parsing power for the words and their dependents: *knowing the identity of the verb can help deciding which noun is the subject or the object.*
- Dependency grammars can handle languages with relatively *free word order*.
- There is no real advantage of dependency grammars over phrase structure grammars and the other way around. Phrase structure grammars are a bit more restricted. Dependency grammars are more flexible. For English it is easier to create a phrase structure grammar, for languages such as Czech (with more word order flexibility) it is easier to create a dependency grammar.
- **Dependency Grammars are very suited for negation handling and co-reference / anaphora resolution** (we will get back to this later in the course).

Teaching note: This is exactly why dependency grammars come back later in this lecture for negation scope and coreference (slides 74, 77–81).

Syntactic Parsing



Maastricht University

Teaching note: Parsing is search: exploring the space of possible trees for one that matches both the grammar and the input sentence.

Syntactic Parsing

- Parsing: combination of recognizing an input string and assigning a structure to it.
- *Syntactic parsing* is recognizing a sentence and assigning a syntactic structure to it.
- Most common parsing algorithms combine top-down and bottom up with principles from dynamic programming (minimal-distance-edit, Viterbi, forward, A^* , ...):
 - CKY (Cocke-Kasami-Younger, 1965)
 - Earley (Earley, 1970)
 - Chart parsing (Kay 1982, Kaplan, 1973)

Parsing is very similar to searching

- **Top-down search:** create each ply in the parse tree by taking each tree from the previous ply, replacing the leftmost non-terminal with each of the possible expansions and collecting each of these trees into a new ply.
- **Bottom-up search:** start with lexicon and then apply grammar rules to build syntactic structure of sentence.

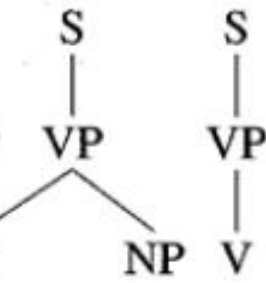
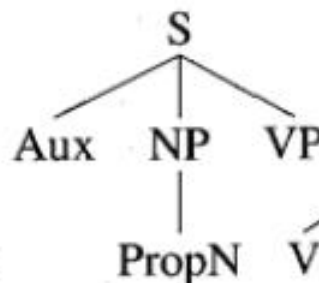
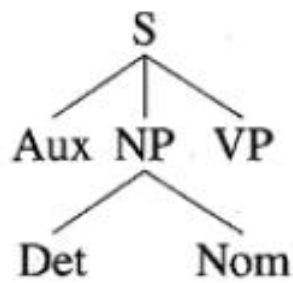
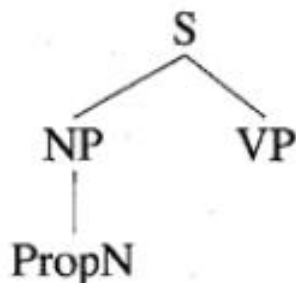
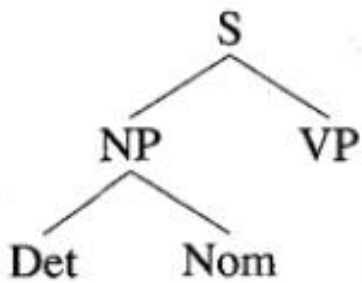
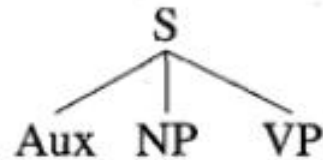
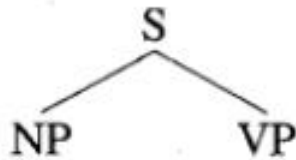
Pro and cons:

- Top-down never wastes time exploring trees that cannot result into a S. Bottom-up does.
- Top-down does spend a lot of time on inconsistent trees. Weakness is that they generate trees before they examine the input. But, in some cases this makes them more robust and allows for use of statistics.
- Combining both strategies with “intelligence” to prune parts of the search tree.
- Very similar to other intelligent search techniques in computer games and AI: dynamic programming.



An Expanding Top-Down Search Space

S



Bottom-up parsing (part one)

Book that flight

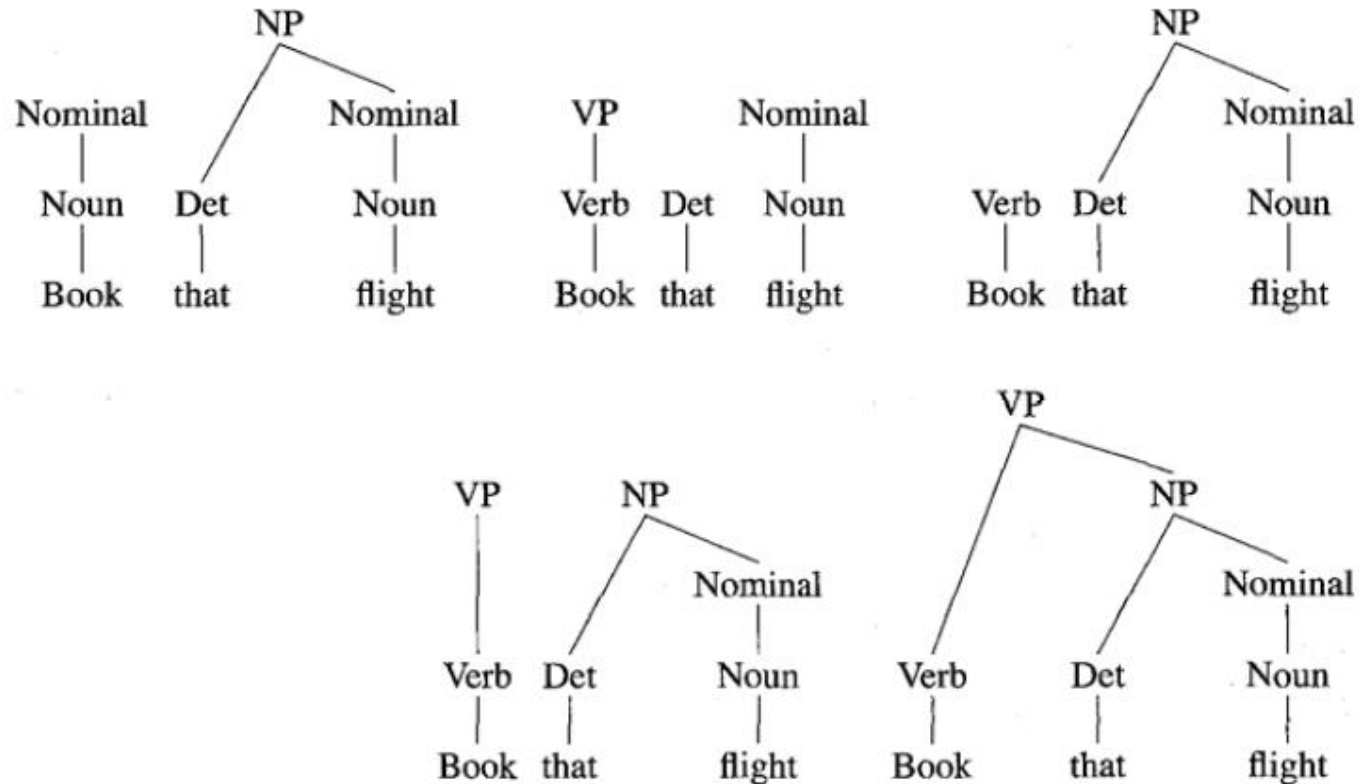
Noun Det Noun
| | |
Book that flight

Verb Det Noun
| | |
Book that flight

Nominal
|
Noun Det Noun
| | |
Book that flight

Nominal
|
Verb Det Noun
| | |
Book that flight

Bottom-up parsing (part two after this the full sentence S will follow (not shown))

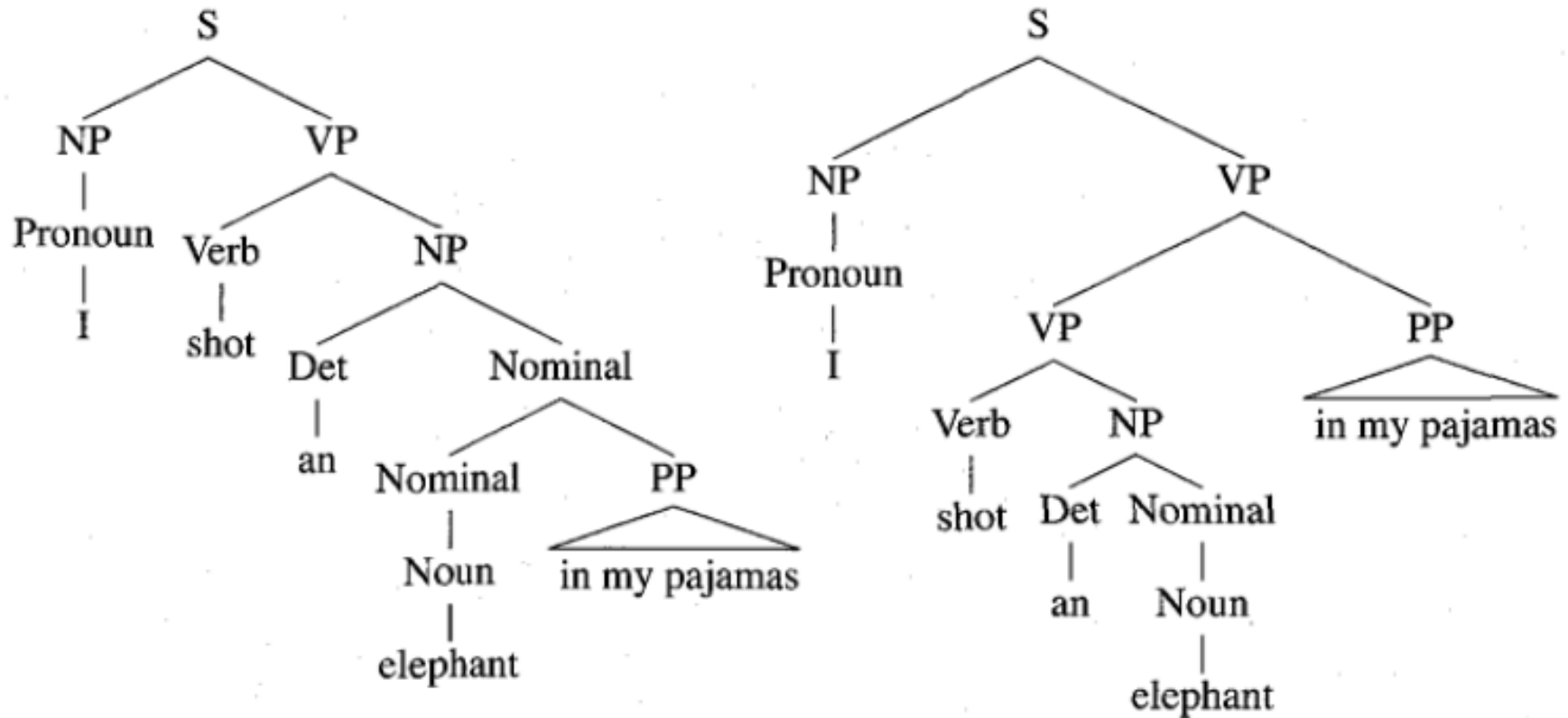


Ambiguity

- Largest problem for parsing is the ambiguity:
 - Structural ambiguity:
 - Attachment ambiguity: a constituent can be attached at multiple locations.
 - Coordination ambiguity: different sets of phrases can be conjoined by a conjunction like *and*. (Old men and women = [[Old men] and women] or [Old [men and women]]).
- Linguistic probability can help to select the most probable parse. But, for ambiguous sentences, one will get multiple parse trees.



Two Parses for an Ambiguous Sentence



Adding Linguistic Probability to CFG Parsing



Maastricht University

Teaching note: Pure CFGs can't choose between competing valid parses — adding probabilities (PCFG) lets the grammar pick the most likely one.

Statistical Parsing

- Used to solve **disambiguation** problem: compute the probability of each interpretation of a sentence parse, then choose the most probable interpretation.
- Used for language modeling in speech recognition and intelligence (email or fax monitoring) in order to deal better with noisy or incorrect input.
- Most common used is the **probabilistic context-free grammar (PCFG)**.
- Other techniques are **Hidden Markov Models, Maximal Entropy Models** and **Conditional Random Fields**. TREEBANKs can be used to train probabilities.
- Statistics can be used to solve all kinds of other ambiguities as well (lexical, sentence boundaries, splitting non-terminals, etc.).



PCFG Definition

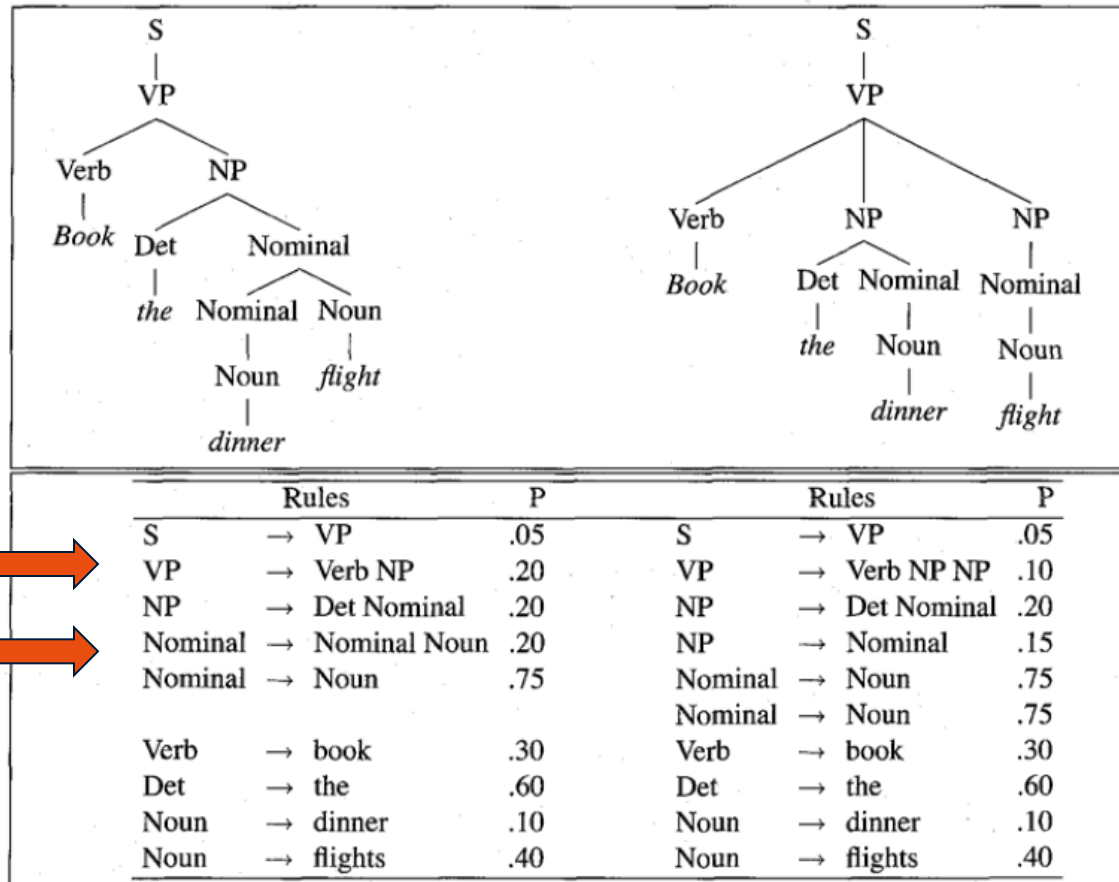
- N a set of **non-terminal symbols** (or **variables**)
- Σ a set of **terminal symbols** (disjoint from N)
- R a set of **rules** or productions, each of the form $A \rightarrow \beta [p]$,
where A is a non-terminal,
 β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$,
and p is a number between 0 and 1 expressing $P(\beta|A)$
- S a designated **start symbol**

$$\sum_{\beta} P(A \rightarrow \beta) = 1$$

PCFG -2

$S \rightarrow NP VP$	[.80]	$Det \rightarrow that [.10] \mid a [.30] \mid the [.60]$
$S \rightarrow Aux NP VP$	[.15]	$Noun \rightarrow book [.10] \mid flight [.30]$
$S \rightarrow VP$	[.05]	$\mid meal [.15] \mid money [.05]$
$NP \rightarrow Pronoun$	[.35]	$\mid flights [.40] \mid dinner [.10]$
$NP \rightarrow Proper-Noun$	[.30]	$Verb \rightarrow book [.30] \mid include [.30]$
$NP \rightarrow Det Nominal$	[.20]	$\mid prefer; [.40]$
$NP \rightarrow Nominal$	[.15]	$Pronoun \rightarrow I [.40] \mid she [.05]$
$Nominal \rightarrow Noun$	[.75]	$\mid me [.15] \mid you [.40]$
$Nominal \rightarrow Nominal Noun$	[.20]	$Proper-Noun \rightarrow Houston [.60]$
$Nominal \rightarrow Nominal PP$	[.05]	$\mid TWA [.40]$
$VP \rightarrow Verb$	[.35]	$Aux \rightarrow does [.60] \mid can [40]$
$VP \rightarrow Verb NP$	[.20]	$Preposition \rightarrow from [.30] \mid to [.30]$
$VP \rightarrow Verb NP PP$	[.10]	$\mid on [.20] \mid near [.15]$
$VP \rightarrow Verb PP$	[.15]	$\mid through [.05]$
$VP \rightarrow Verb NP NP$	[.05]	
$VP \rightarrow VP PP$	[.15]	
$PP \rightarrow Preposition NP$	[1.0]	

Disambiguating with statistics: “book a flight that serves dinner” versus “book a flight on behave of “the dinner””



There are many annotated corpora to derive linguistic probabilities

- The Brown Corpus was the first large annotated corpus.
- The [Brown Corpus](#) has also spawned a number of similarly structured corpora: the [LOB Corpus](#) (1960s [British English](#)), Kolhapur ([Indian English](#)), Wellington ([New Zealand English](#)), Australian Corpus of English ([Australian English](#)), the Frown Corpus ([early 1990s American English](#)), and the FLOB Corpus (1990s British English). Other corpora represent many languages, varieties and modes, and include the [International Corpus of English](#), and the [British National Corpus](#), a 100 million word collection of a range of spoken and written texts, created in the 1990s by a consortium of publishers, universities ([Oxford](#) and [Lancaster](#)) and the [British Library](#). For contemporary American English, work has stalled on the [American National Corpus](#), but the 400+ million word [Corpus of Contemporary American English](#) (1990–present) is now available through a web interface.

But there is more to language ...

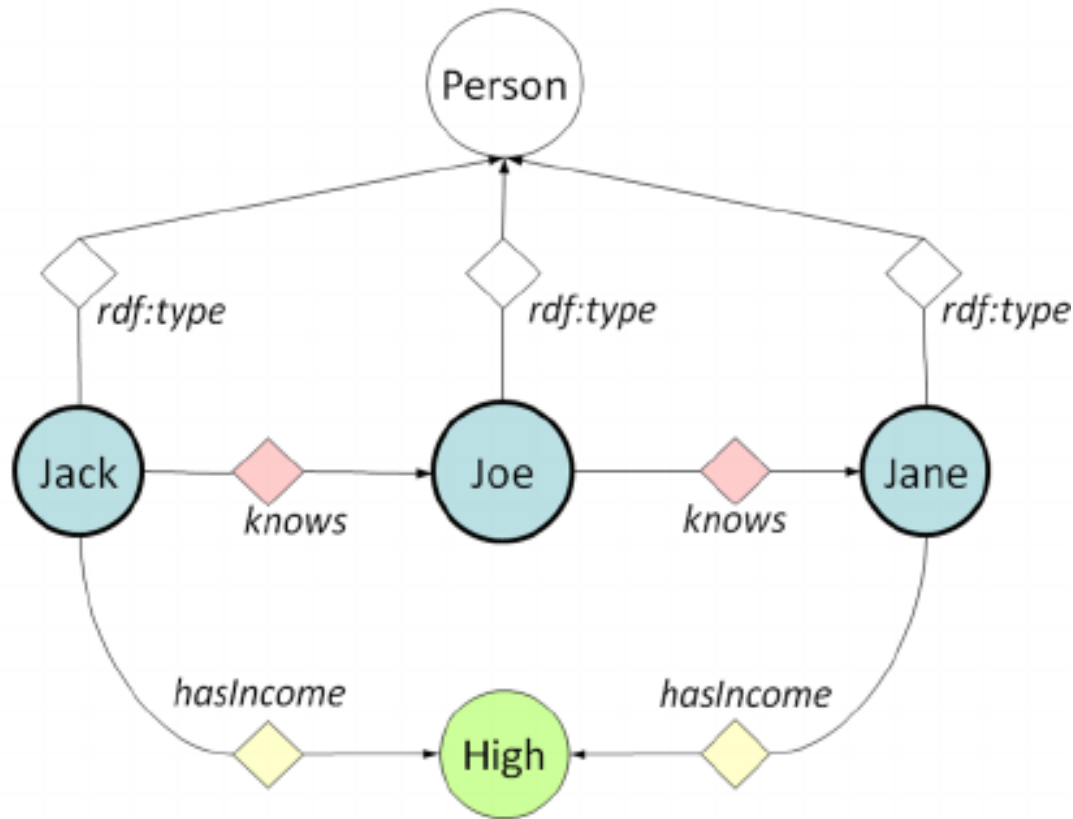
Co-references, anaphora,
pronouns & negations

Natural Language Processing

- **Semantics:** In linguistics, semantics is the subfield that is devoted to the **study of meaning**, as inherent at the levels of words, phrases, sentences, and larger units of discourse (referred to as texts). The basic area of study is the meaning of signs, and the study of relations between different linguistic units.
- A key concern is how meaning attaches to larger chunks of text, possibly as a result of the composition from smaller units of meaning.

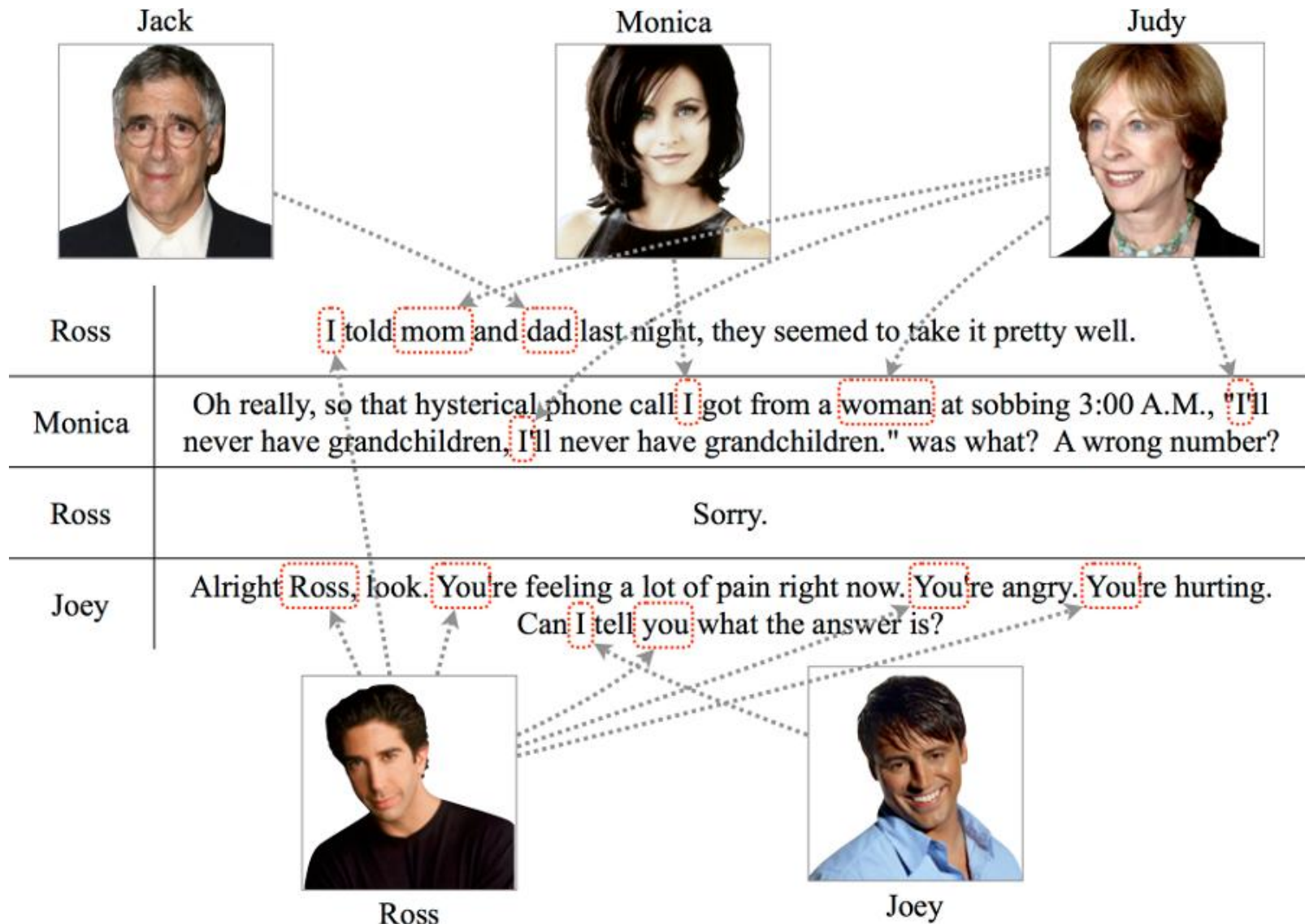


Semantic Analysis with RDF*



RDF = Resource Description Framework – will be discussed in Course Information Retrieval and Text-Mining in more detail

Co-reference & Anaphora Resolution



Anaphora and co-references - 2

- *Pronominal Anaphora*: he, she, we, oneself, etc.
- *Proper Name Co-reference*: For example, multiple references to the same name.
- *Apposition*: the additional information given to an entity, such as “Jan Jansen, the father of Piet Jansen”.
- *Predicate Nominative*: the additional description given to an entity, for example “Jan Jansen, who is the chairman of the football club”.
- *Identical Sets*: A number of reference sets referring to equivalent entities, such as “Ajax”, “the Amsterdam team”, and the “group of players” which all refer to the same group of people.

Anaphora and co-references - 3

- *Function value co-reference*: characterized by phrases that have a function-value relationship. Typically, the function will be descriptive and the value numeric: “ABC Ltd revenue’s were \$ 4 million for the quarter, up 61% compared to the same quarter in the previous year. *Net loss* declined by 34% to *\$ 1 million*.”
- *Ordinal Anaphora*: involved cardinal numbers such as *first*, *second*, or an adjective such as *former* or *latter*. “Dell and IBM were the final candidates, but the customer preferred the first.”
- *One-anaphora*: anaphora expressions that are build of a noun phrase that contains the word “one” as in: “if you cannot attend a tutorial in the morning, you can follow the one in the afternoon.”
- *Whole part co-reference*: when an anaphora refers to a part of the antecedent such as in: “John bought a new car. The indicators use the latest laser technology.”

Constraints used in Co-reference and Anaphora Resolutions

1. Gender agreement: Antecedents which do not agree in terms of their gender, need not be considered further for evaluation of their correctness.
2. Number agreement: An entity may corefer with another entity if and only if they agree on the basis of their singularity and plurality.
3. Person agreement: The co-referent nouns or entities must agree with respect to their person (I/me, you, he/she/it, we/our, they).
4. Grammatical role: Entities occurring in the subject portion of a sentence are given a higher priority than the entity in object position.
5. Recency: Entities introduced recently have more salience than entities which have occurred earlier in the discourse.
6. Repeated mention: Entities which have been introduced repeatedly in the context or have been the main focus or topic prevail.
7. Discourse structure: The preference of one entity over another can also be due to the structural idiosyncrasies of the discourse like parallelism. “Aryan(1) passed a note to Frank(2) and John(3) gave him (4) a chocolate”, (4) refers to (2) and not (1) due to the discourse structure involved here.
8. World knowledge: Commonsense concepts like “cat-meows” and “dog-barks” cannot be resolved only by studying the syntactic properties.

Teaching note: Notice the last constraint (world knowledge) can't be solved by any syntactic rule — this is exactly why deep learning eventually overtakes rule-based resolution (slide 75).

Approaches to Anaphora Resolution

- Most common approach is by using pronominal resolution because that is the most common anaphor and that is also easier to solve. The approach is as follows:
 - Identify the relevant paragraphs (or sentences) around the pronoun in which one will search for candidate antecedents.
 - Using a set of consistency checks, delete the candidates that do not meet any of the checks.
 - Assign salience (=most noticeable or important) values to each of the surviving candidates according to a set of rules.
 - Pick the candidate with the highest salience value.
- Some approaches rely heavily on deep parsing, others are knowledge-poor and use shallow parsing: hand-crafted rules and heuristics, dependency grammars & (recently) LSTMs.
- Modern approaches use Dependency Grammars or Deep Learning

Hobbs Algorithm (1986)

- Aka the *naïve* algorithm for anaphora resolution.
- Works by specifying a total order of noun phrases in the prior discourse of and comparing each noun phrase against a set of constraints imposed by features of the anaphor such as gender, number, etc.
- The first antecedent to satisfy all constraints is chosen.
- Remarks:
 - For 2 candidate antecedents a and b : if a is encountered in the search space before b , then a is chosen.
 - The salience given to the candidate antecedents, imposes a total ordering on the antecedents: no two antecedents will have the same salient.
 - Cannot handle ambiguity. Pronouns are resolved as if there were at least one possible antecedent.

Evaluation of Knowledge Poor Approaches

- For many years there was no common ground to compare anaphora resolution algorithms and approaches.
- Barbu and Mitkov created an evaluation workbench in 2001.
- Results were as follows:

	K&B	Cogniac	Mitkov
Precision	52.84%	42.65%	48.81%
Success	61.6%	49.72%	56.9%

THESE ARE NOT VERY GOOD RESULTS, SO WE NEEDED SOMETHING
DIFFERENT



Dependency Grammars for Co-reference & Pronoun Resolution are better than the former...

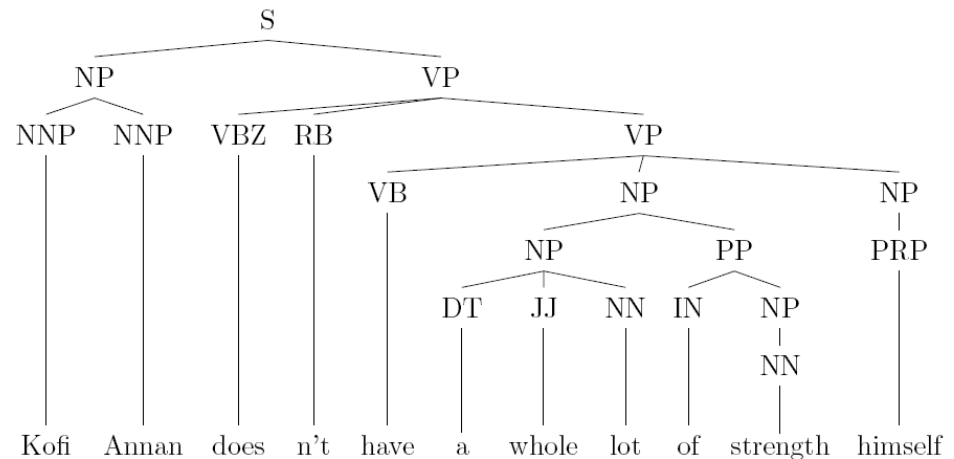


Figure 1: An example phrase structure tree.

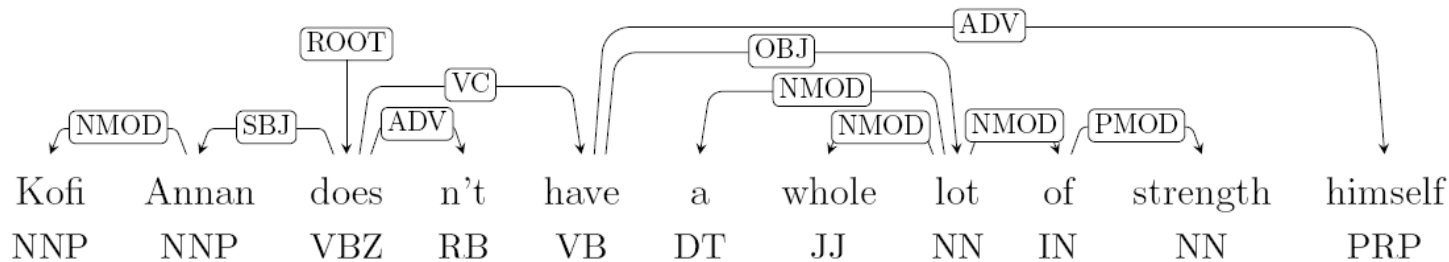


Figure 2: Dependency representation of the example from Figure 1.

Source: Phrase Structures and Dependencies for End-to-End Coreference Resolution, Anders Bjorkelund Jonas Kuhn, Institut für Maschinelle Sprachverarbeitung, University of Stuttgart

State of the Art for Co-reference & Anaphora Handling is in Deep learning

Model	MUC			B ³			CEAF _{ϕ_4}			Avg. F1
	P	R	F1	P	R	F1	P	R	F1	
Clark and Manning (2015)	76.1	69.4	72.6	65.6	56.0	60.4	59.4	53.0	56.0	63.0
Wiseman et al. (2016)	77.5	69.8	73.4	66.8	57.0	61.5	62.1	53.9	57.7	64.2
Clark and Manning (2016)	79.2	70.4	74.6	69.9	58.0	63.4	63.5	55.5	59.2	65.7
Lee et al. (2017)	78.4	73.4	75.8	68.6	61.8	65.0	62.7	59.0	60.8	67.2
Fei et al. (2019)	85.4	77.9	81.4	77.9	66.4	71.7	70.6	66.3	68.4	73.8
Xu and Choi (2020)	85.9	85.5	85.7	79.0	78.9	79.0	76.7	75.2	75.9	80.2
Wu et al. (2020)	88.6	87.4	88.0	82.4	82.0	82.2	79.9	78.3	79.1	83.1
Baseline (Lee et al., 2018)	81.4	79.5	80.4	72.2	69.5	70.8	68.2	67.1	67.6	73.0
+ BERT-base (Joshi et al., 2019)	80.4	82.3	81.4	69.6	73.8	71.7	69.0	68.5	68.8	73.9
+ BERT-large (Joshi et al., 2019)	84.7	82.4	83.5	76.5	74.0	75.3	74.1	69.8	71.9	76.9
+ SpanBERT-large (Joshi et al., 2020)	85.8	84.8	85.3	78.3	77.9	78.1	76.4	74.2	75.3	79.6
G2GT BERT-base <i>truncated</i>	78.4	77.9	78.1	69.6	71.0	70.3	66.8	67.3	67.0	71.8
G2GT BERT-base <i>overlap</i>	81.2	82.8	82.0	69.8	73.6	71.6	69.6	69.3	69.4	74.4
G2GT BERT-base <i>reduced</i>	83.4	83.1	83.2	70.1	73.7	71.9	72.1	70.1	71.0	75.4
G2GT BERT-large <i>truncated</i>	80.1	79.2	79.6	71.3	71.0	71.1	69.1	68.8	68.9	73.2
G2GT BERT-large <i>overlap</i>	83.5	83.2	83.3	74.5	74.1	74.3	75.2	70.1	72.6	76.7
G2GT BERT-large <i>reduced</i>	84.7	83.1	83.9	76.8	74.0	75.4	75.3	70.1	72.6	77.3
G2GT SpanBERT-large <i>overlap</i>	85.8	84.9	85.3	78.7	78.0	78.3	76.4	74.5	75.4	79.7
G2GT SpanBERT-large <i>reduced</i>	85.9	86.0 * [†]	85.9 *	79.3 *	79.4 * [†]	79.3 *	76.4	75.9 *	76.1 *	80.5 *

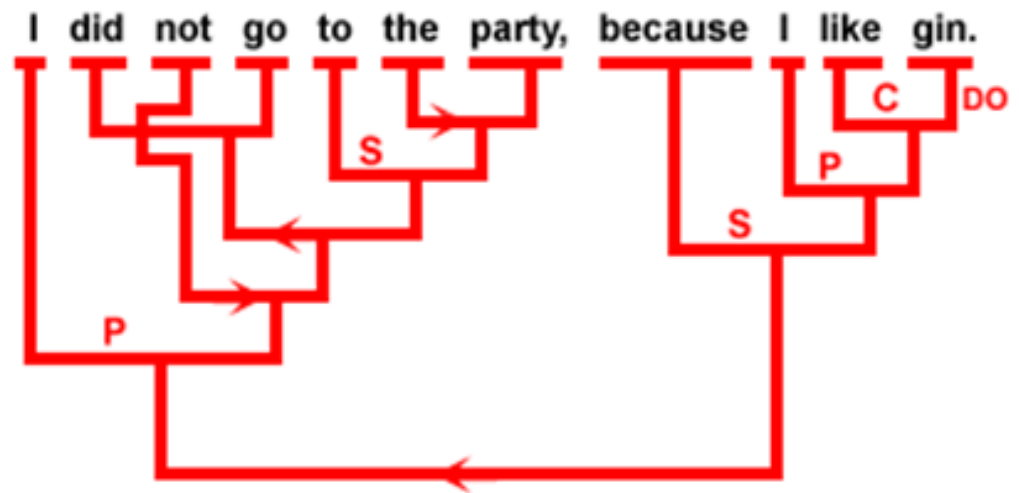
Table 3: Evaluation on the test set (CoNLL 2012). * significant at $p < 0.01$ compared to (Joshi et al., 2020), [†] significant at $p < 0.05$ compared to (Xu and Choi, 2020)

As we will see later in this course, Deep Learning models are the SOTA for this task. See also: <https://arxiv.org/pdf/2203.16574.pdf> and <https://arxiv.org/pdf/2303.09092.pdf>



Negation Handling

~~impossible~~
~~impatient~~
~~unsafe~~
~~unknown~~
~~unemployed~~



Handling Negations with Dependency Grammars

There are many different forms of negation. Two basic categories can be identified:

- Morphological

- A negating prefix such as dis-, non-, un- or suffix such as –less.
- Scope restricted to individual word.
- Easy to recognize and handle.

- Syntactical

- Uses explicit negation words and syntactic patterns that imply negative meaning.
- Determining scope and strength are complex.



Syntactic Negations

- Denials
 - “there is no question that the service is” or “the audio in this TV is not very good”.
- Rejections
 - “Can I get you something else?. No.” or “given the poor reputation of the manufacturer, I expected to be disappointed. This was not the case”.
- Imperatives
 - “do not neglect to order their delicious garlic bread”.
- Questions
 - “Why couldn’t they include a better speaker”.
- Supports and repetitions
 - Express agreement and add emphasis or clarity, respectively, and each involve multiple expressions of negation.

Scope of a Negation

- Inter-sentential
 - Rejections
 - Supports
 - Are less frequent
- Sentential
 - Denials
 - Imperatives
 - Questions
 - Are more frequent

Shifting windows and n-grams do not result in
reliable scope determination, but
Dependency Grammars can!

```

(ROOT
  (S
    (S
      (NP (PRP I))
      (VP (VBD thought)
        (SBAR
          (S
            (NP (DT the) (JJ first) (NN season))
            (VP (VBD was)
              (ADJP (JJ outstanding))))))
      (, ,)
    )
    (S
      (NP (DT the) (JJ second) (NN season))
      (VP (VBD was)
        (ADJP (JJ mediocre)
          (PP (IN at)
            (ADJP (JJS best))))))
      (CC and)
    )
    (S
      (NP (DT the) (JJ third) (NN season))
      (VP (VBD was)
        (ADJP (RB no) (JJ good))
        (PP (IN at)
          (NP (DT all))))))
    (. .)))

```

Figure 1: An example parse tree produced by the Stanford parser

Tag	Part of speech
CC	Coordinating conjunction
CD	Cardinal number
DT	Determiner
EX	Existential <i>there</i>
FW	Foreign word
IN	Preposition or subordinating conjunction
JJ	Adjective
JJR	Adjective, comparative
JJS	Adjective, superlative
LS	List item marker
MD	Modal
NN	Noun, singular or mass
NNS	Noun, plural
NNP	Proper noun, singular
NNPS	Proper noun, plural
PDT	Predeterminer
POS	Possessive ending
PRP	Personal pronoun
PRP\$	Possessive pronoun
RB	Adverb
RBR	Adverb, comparative
RBS	Adverb, superlative
RP	Particle
SYM	Symbol
TO	<i>to</i>
UH	Interjection
VB	Verb, base form
VBD	Verb, past tense
VBG	Verb, gerund or present participle
VBN	Verb, past participle
VBP	Verb, non-3rd person singular present
VBZ	Verb, 3rd person singular present
WDT	Wh-determiner
WP	Wh-pronoun
WP\$	Possessive wh-pronoun
WRB	Wh-adverb

Table 1: Penn Treebank part of speech tags

Another example of Dependency Grammar Result Representation (from Wilson et al.,).

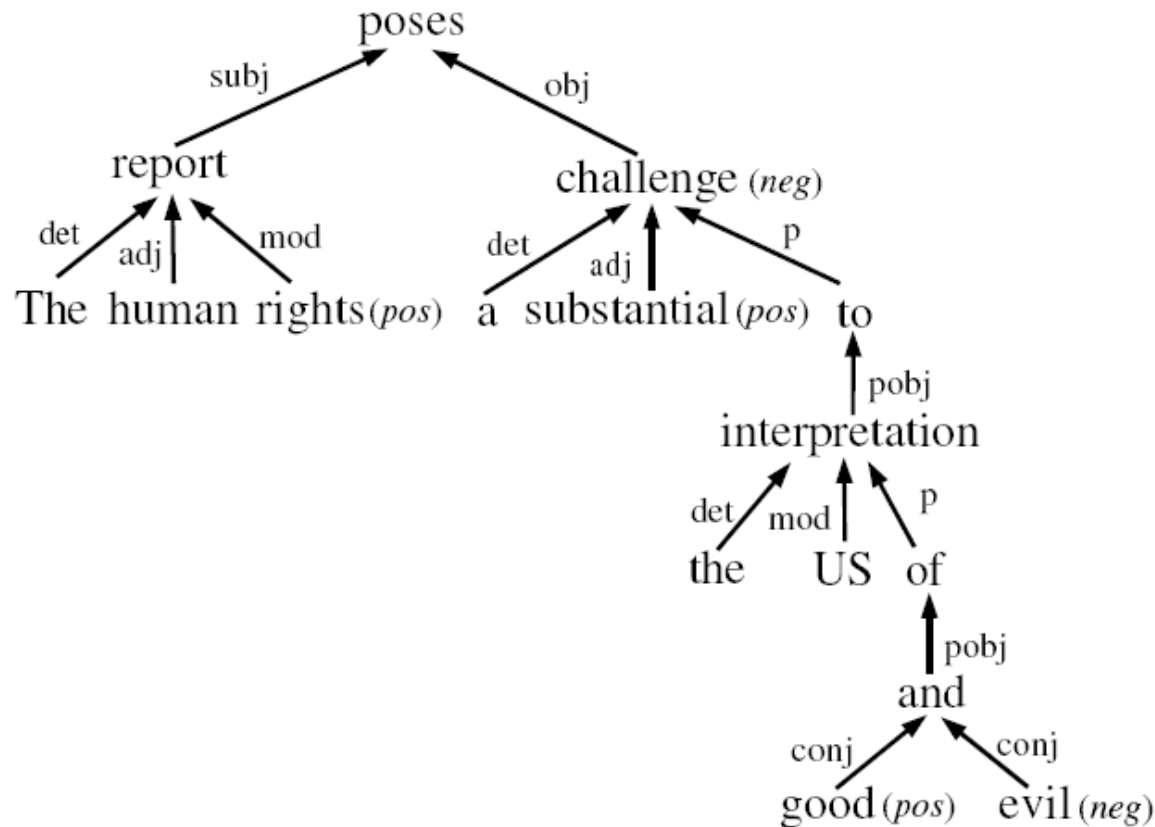


Figure 1: The dependency tree for the sentence *The human rights report poses a substantial challenge to the US interpretation of good and evil*. Prior polarity is marked in parentheses for words that match clues from the lexicon.

Negation Handling & Deep Learning: not a complete success ... source of many forms of hallucinations ...

Context	BERT _{LARGE} predictions
<i>A robin is a ____</i>	<i>bird, robin, person, hunter, pigeon</i>
<i>A daisy is a ____</i>	<i>daisy, rose, flower, berry, tree</i>
<i>A hammer is a ____</i>	<i>hammer, tool, weapon, nail, device</i>
<i>A hammer is an ____</i>	<i>object, instrument, axe, implement, explosive</i>
<i>A robin is not a ____</i>	<i>robin, bird, penguin, man, fly</i>
<i>A daisy is not a ____</i>	<i>daisy, rose, flower, lily, cherry</i>
<i>A hammer is not a ____</i>	<i>hammer, weapon, tool, gun, rock</i>
<i>A hammer is not an ____</i>	<i>object, instrument, axe, animal, artifact</i>

Table 13: NEG-88-SIMP predictions by BERT_{LARGE}

In the lecture on BERT we will discuss this phenomena and possible solutions in more detail.

Take Away

- The methods discussed in this course are traditional computational linguistics.
- Not really fault tolerant.
- Hard to create and maintain.
- Not very successful for real-world applications.

However, when we get to transformers, all the linguistic phenomena we discussed today will return and you will understand why transformers are as good as they are dealing with human language.

Questions?



Maastricht University

Exercise 1

Slide 25 demonstrates recursive center-embedding in natural language: *"The luggage arrived"* → *"The luggage that the passengers checked arrived"* → *"The luggage that the passengers that the storm delayed checked arrived"*.

- Using formal language theory and the Chomsky hierarchy (Regular, Context-Free, Context-Sensitive, Recursively Enumerable), explain mathematically why a Finite State Automaton (FSA) or regular grammar cannot generate center-embedded structures of arbitrary depth.
- Why does recursive center-embedding require at least a Context-Free Grammar (CFG) / push-down automaton?
- Consider the simplified grammar L_0 from the lecture:
 - $S \rightarrow NP VP$
 - $NP \rightarrow Det Nominal \mid Pronoun$
 - $Nominal \rightarrow Noun \mid Nominal Noun$
 - $VP \rightarrow Verb NP \mid Verb$
 - Given the lexicon: $Det \rightarrow a$, $Noun \rightarrow flight \mid morning$, $Pronoun \rightarrow I$, $Verb \rightarrow want$.
 - Draw or write out the step-by-step derivation for the sentence: *"I want a morning flight"*.
Where does noun stacking occur, and how does the CFG rule accommodate it?

Exercise 2

Compare Phrase-Structure (Constituency) grammars and Dependency grammars in their treatment of sentence structure and structural ambiguity.

- Contrast the structural representation of a sentence in a Constituency parse versus a Dependency parse:
 - What serves as the root node in each framework?
 - How do the two paradigms differ regarding non-terminal phrasal nodes (e.g., *NP*, *VP*, *PP*)?
 - Why are dependency grammars more naturally suited for free-word-order languages (such as Czech or Dutch) than strict phrase-structure grammars?
- Consider the structurally ambiguous sentence: *"I saw the man on the hill with a telescope"*. Explain the difference between **attachment ambiguity** and **coordination ambiguity**.
- In a Probabilistic Context-Free Grammar (PCFG), the probability of a parse tree is computed by multiplying the probabilities of all independent rule expansions used. State the primary independence assumption made by a PCFG rule, and explain why this independence assumption fails to capture lexical preferences between specific verbs and prepositions

Exercise 3

Complex Discourse Phenomena (Coreference, Heuristics, and Negation Scope). Beyond sentence-level syntax, systems must resolve non-local discourse relations, including pronominal anaphora and negation scope.

- In the Hobbs (1986) algorithm and classical knowledge-poor anaphora resolution systems, candidates are filtered and ranked using grammatical constraints and salience heuristics.
 - List three agreement constraints (e.g., gender, number, person) used to prune invalid candidates.
 - If two surviving candidate nouns both satisfy all agreement checks, what ordering and structural heuristics (e.g., grammatical role, recency) determine which antecedent is chosen?
- Distinguish between **morphological negation** (e.g., *impossible*, *unemployed*) and **syntactic negation** (e.g., *denials*, *rejections*, *imperatives*).
- In the sentence: "*The audio in this TV is not very good*":
 - Why do simple fixed sliding windows and n-grams fail to reliably determine the exact scope of the negation?
 - Explain how traversing a **dependency grammar parse tree** allows an NLP system to locate the precise subtree/dependent modified by the negation marker (neg).

Discussion Lecture: Syntax Semantics



Maastricht University

Teaching note: Use today's grammar formalisms (CFG, dependency grammar, negation scope) as concrete cases for the abstract questions ahead.

On Chomsky's Hierarchy: Is Natural Language Truly Context-Free?

In formal language theory, we learn that finite state automata fail on natural language because they lack memory for arbitrary recursive center-embedding, requiring us to jump to Context-Free Grammars (CFGs).

Yet when humans speak Dutch or Swiss German, cross-serial dependencies emerge that mathematically violate context-freeness entirely. If human grammar routinely breaks context-free boundaries, why has computational linguistics spent decades approximating syntax with CFGs, trees, and push-down automata?

Are we forcing human thought into convenient mathematical automata, or is formal syntax just an engineering scaffold that natural language constantly outgrows?

When we are further in this course, this is an interesting question to look at in relation to the Transformer architecture.



On Constituency vs. Dependency: What is the Fundamental Unit of Syntax?

Constituency grammars treat sentences as nested, hierarchical boxes of abstract phrases (*NP*, *VP*, *PP*), while Dependency grammars discard non-terminals altogether, anchoring every connection directly between words around a central verb root.

If both models claim to capture the underlying structure of the exact same sentence, which representation reflects how human cognition organizes language?

Why did modern NLP pipelines (like spaCy and Universal Dependencies) overwhelmingly abandon Chomskyan constituency trees in favor of dependency graphs when solving practical tasks like relation extraction and negation scope



On Negation and Deep Learning: Why Do Modern LLMs Still Stumble?

We saw that classical dependency parsers handle complex syntactic negation by pinpointing the exact syntactic subtree governed by a negative particle.

Yet psycholinguistic diagnostics show that modern deep learning models (even large transformers) frequently fail to process negation correctly, treating 'A is not B' as highly similar to 'A is B' because both share identical topical representations. If end-to-end transformers possess billions of parameters and capture complex context, why do they remain notoriously vulnerable to the simple insertion of the word 'not'?

Did our shift from symbolic dependency trees to continuous vector spaces solve syntax, or did it blind us to strict propositional logic?

References

- Most of the topics discussed in this lecture are also explained in: Speech and Language Processing (Dan Jurafsky and James H. Martin). Prentice Hall, 2021.

<https://web.stanford.edu/~jurafsky/slp3/>



More (detailed) References

- Chomsky, N. (1957). Syntactic Structures. The Hague Mouton.
- For POS tagging: Foundations of Statistical Natural Language Processing. Christopher D. Manning and Hinrich Schütze.
Chapter 10: Part-of-Speech Tagging
- For Dependency Grammars: Speech and Language Processing (Dan Jurafsky and James H. Martin). Prentice Hall, 2021.
<https://web.stanford.edu/~jurafsky/slp3/>, Chapter 14.
- For Co-reference resolution: Speech and Language Processing (Dan Jurafsky and James H. Martin). Prentice Hall, 2021.
<https://web.stanford.edu/~jurafsky/slp3/>, chapter 21.